

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

API 函数自搜索

实验要求：

复现第五章实验七，基于示例 5-11，完成 API 函数自搜索的实验，将生成的 exe 程序，复制到 windows 10 操作系统里验证是否成功。

实验过程：

前面我们讲的 Shellcode 的编写，都采用硬编址的方式来调用相应 API 函数。先获取所要使用函数的地址，然后将该地址写入 ShellCode，从而实现调用。如果系统的版本变了，函数的地址往往都会发生变化，那么调用就会失败。

为了编写通用 shellcode, shellcode 自身就必须具备动态的自动搜索所需 API 函数地址的能力，即 API 函数自搜索技术。

1. 通用型 Shellcode 的编写逻辑

以 MessageBoxA 函数的调用的 shellcode 为例，来解释通用型 shellcode 的编写逻辑。

首先分析一下用到的函数：

- MessageBoxA 位于 user32.dll 中，用于弹出消息框。
- ExitProcess 位于 kernel32.dll 中，用于正常退出程序。所有的 Win32 程序都会自动加载 ntdll.dll 以及 kernel32.dll（两个最基础的动态链接库）。
- LoadLibraryA 位于 kernel32.dll 中，在调用 MessageBoxA 之前，应该先使用 LoadLibrary 装载 user32.dll。

那么，通用型 shellcode 编写的大致步骤可以概括为：

- （1）第一步：定位 kernel32.dll。
- （2）第二步：定位 kernel32.dll 的导出表。
- （3）第三步：搜索定位 LoadLibrary 等目标函数。
- （4）第四步：基于找到的函数地址，完成 Shellcode 的编写。

2. API 函数自搜索技术

1>定位 kernel32.dll

在 Win32 平台下定位 kernel32.dll 中的 API 地址，方法大致如下：

- （1）首先通过段选择字 FS 在内存中找到当前的线程环境块 TEB。
- （2）线程环境块偏移地址为 0x30 的地址存放着指向进程环境块 PEB 的指针。
- （3）进程环境块中偏移地址为 0x0c 的地方存放着指向 PEB_LDR_DATA 结构体的指针，其中，存放着已经被进程装载的动态链接库的信息。
- （4）PEB_LDR_DATA 结构体偏移位置为 0x1C 的地址存放着指向模块初始化链表的头指针。
- （5）模块初始化链表中按顺序存放着 PE 装入运行时初始化模块的信息，第一个链表结点是

ntdll.dll，第二个链表结点就是 kernel32.dll。

(6) 找到属于 kernel32.dll 的结点后，在其基础上再偏移 0x08 就是 kernel32.dll 在内存中的加载基地址。

```
int main()
{
    _asm
    {
        mov eax, fs:[0x30] ;PEB 的地址
        mov eax, [eax + 0x0c] ; PEB_LDR_DATA 结构体的地址
        mov esi, [eax + 0x1c] ; 指针 InInitializationOrderModuleList
        lodsd
        mov eax, [eax + 0x08] ;eax 就是 kernel32.dll 的地址
    } return 0;
}
```

2>定位 kernel32.dll 的导出表

kernel32.dll 属于 PE 文件，根据 PE 文件的结构特征，定位其导出表，进而定位导出函数列表信息，然后进行解析、遍历搜索，可以找到我们所需要的 API 函数。步骤如下：

- (1) 从 kernel32.dll 加载基址算起，偏移 0x3c 的地方就是其 PE 头的指针。
- (2) PE 头偏移 0x78 的地方存放着指向函数导出表的指针。
- (3) 获得导出函数偏移地址（RVA）列表、导出函数名列表。

```
mov     ebp, eax           //将 kernel32.dll 基地址赋值给 ebp
mov     eax, [ebp+0x3C]    //dll 的 PE 头的指针（相对地址）
mov     ecx, [ebp+eax+0x78] //导出表的指针（相对地址）
add     ecx, ebp           //ecx=0x78C00000+0x262c 得到导出表的内存地址
mov     ebx, [ecx+0x20]    //导出函数名列表指针
add     ebx, ebp           //导出函数名列表指针的基地址
```

3>搜索定位目标函数

通过遍历两个函数相关列表，可以算出所需函数的入口地址：

- (1) 函数的 RVA 地址和名字按照顺序存放在上述两个列表中，我们可以在名称列表中定位到所需的函数是第几个，然后在地址列表中找到对应的 RVA。
- (2) 获得 RVA 后，再加上前边已经得到的动态链接库的加载地址，就获得了所需 API 此刻在内存中的虚拟地址，这个地址就是最终在 ShellCode 中调用时需要的地址。

这样，可以获得 kernel32.dll 中的任意函数。

为了让 shellcode 更加通用，一般情况下并不会以长字符串去进行直接比较。所以会对所需的 API 函数名进行 hash 运算，这样只要比较 hash 所得摘要就能判定是不是我们所需的 API 了。

```
#include <stdio.h>
#include <windows.h>
DWORD GetHashCode(char *fun_name)
{
    DWORD digest=0;
    while(*fun_name)
    {
```

```

        digest=((digest<<25)|(digest>>7)); //循环右移 7 位
/*    movsx    eax,byte ptr[esi]
        cmp     al,ah
        jz      compare_hash
        ror     edx, 7 ; ((循环))右移,不是单纯的 >>7
        add     edx,eax
        inc     esi
        jmp     hash_loop
*/
    digest+= *fun_name ; //累加
    fun_name++;
}
return digest;
}
main()
{
    DWORD hash;

    hash= GetHashCode("MessageBoxA");

    printf("%#x\n", hash);
}

```

通过上述代码，我们可以获得 MessageBoxA 的 hash 值。接下来，我们可以在 shellcode 中通过压栈的方式将这个 hash 值压入栈中，再通过比较得到动态链接库中的 API 地址。如下图所示：

<pre> CLD //清空标志位DF push 0x1E380060 //压入MessageBoxA的hash-->user32.dll push 0x4FD18963 //压入ExitProcess的hash-->kernel32.dll push 0x0C917432 //压入LoadLibraryA的hash-->kernel32.dll mov esi,esp lea edi,[esi-0xc] //esi=esp,指向堆栈中存放LoadLibraryA的hash的共 //空出8字节应该也是为了兼容性 //=====是否找到了自己所需全部的函数 Find_lib_functions: lodsd //即move eax,[esi], esi++,第一次取LoadLibraryA的hash cmp eax,0x1E380060 //与MessageBoxA的hash比较 jne find_functions //如果没有找到MessageBoxA函数,继续找 xchg eax,ebp //-----> call [edi+0x8] //LoadLibraryA("user32") xchg eax,ebp //ebp=user32.dll的基地址,eax=MessageBoxA的hash <-- //=====导出函数名列表指针 find_functions: pushad //保护寄存器 mov eax,[ebp+0x3C] //dll的PE头 mov ecx,[ebp+eax+0x78] //导出表的指针 add ecx,ebp //ecx=导出表的基地址 mov ebx,[ecx+0x20] //导出函数名列表指针 add ebx,ebp //ebx=导出函数名列表指针的基地址 xor edi,edi //=====找下一个函数名 next_function_loop: inc edi mov esi,[ebx+edi*4] //从列表数组中读取 add esi,ebp //esi = 函数名称所在地址 cdq </pre>	<pre> //=====函数名的hash运算 hash_loop: movsx eax,byte ptr[esi] cmp al,ah //字符串结尾就跳出当前函数 jz compare_hash ror edx,7 add edx,eax inc esi jmp hash_loop //=====比较找到的当前函数的hash是否是自己想找的 compare_hash: cmp edx,[esp+0x1C] //lods pushad后,栈+1c为LoadLibraryA的hash jnz next_function_loop mov ebx,[ecx+0x24] //ebx = 顺序表的相对偏移量 add ebx,ebp //顺序表的基地址 mov di,[ebx+2*edi] //匹配函数的序号 mov ebx,[ecx+0x1C] //地址表的相对偏移量 add ebx,ebp //地址表的基地址 add ebp,[ebx+4*edi] //函数的基地址 xchg eax,ebp //eax<=>ebp 交换 pop edi stosd //把找到的函数保存到edi的位置 push edi popad cmp eax,0x1E380060 //找到最后一个函数MessageBox后,跳出循环 jne Find_lib_functions </pre>
--	--

4>基于函数地址，完成 shellcode 代码的编写

要输出“westwest”，编写以下 shellcode 代码：

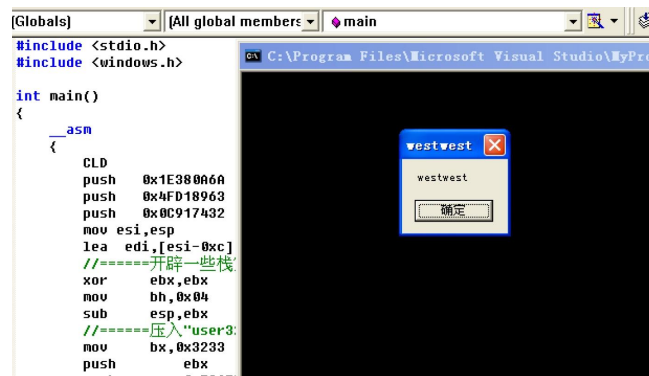
```

function_call:
    xor     ebx,ebx
    push    ebx
    push    0x74736577 //push "westwest"
    mov     eax,esp
    push    ebx
    push    eax
    push    eax
    push    ebx
    call    [edi-0x04] //MessageBoxA(NULL,"westwest","westwest",NULL)
    push    ebx
    call    [edi-0x08] //ExitProcess(0);
    nop
    nop
    nop
    nop

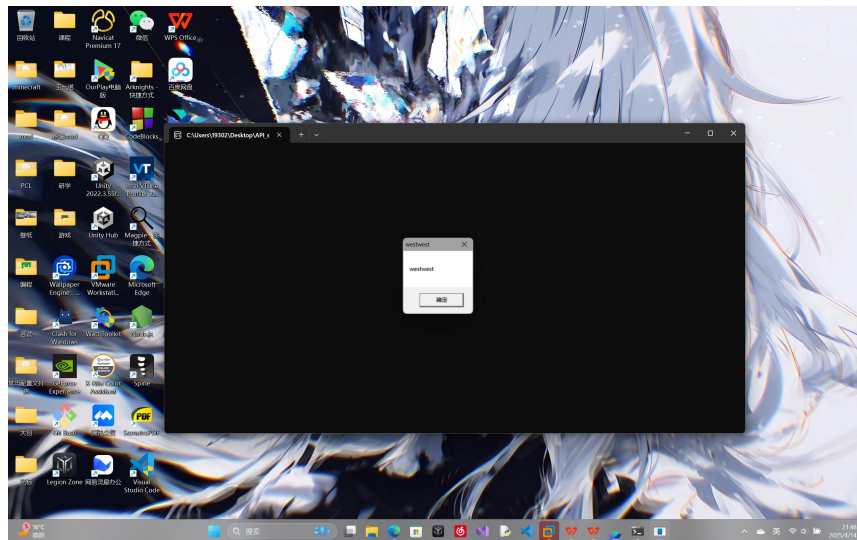
```

3. 验证

将上述所有步骤综合写出完整代码，在 VC6 运行结果如下：



说明成功找到了函数地址并运行了 shellcode。接下来，将生成的 exe 文件转移到自己的电脑即 Win11 系统下，运行结果如下，证明了编写的 shellcode 在不同系统上都能实现 API 自搜索。



心得体会：

- 1.通过本次实验，更具体、更深入地了解了 API 函数的自搜索技术。学会了通过 TEB、PEB 等定位 kernel32.dll，定位导出表。
- 2.理解了通用性 shellcode 的编写逻辑。首先，定位 kernel32.dll，定位导出表，接着定位目标函数，然后根据得到的函数地址，便可编写 shellcode 代码。
- 3.对汇编语言、对 shellcode 有了更深的认识和理解，同时对寄存器的使用操作有了更深的了解。